

25 questions pièges en entretien technique



Réponse tentante vs réponse attendue.

Un entretien Data Analyst junior ne se joue pas sur ce que tu sais réciter. Il se joue sur ces 25 questions précises, où une réponse semble évidente et où une autre, plus juste, est réellement attendue. Un piège n'est pas une trahison du recruteur : c'est un signal qu'il t'envoie sur ta façon de penser. Pour chacune des 25 questions, tu as la réponse qui tente, pourquoi elle tente, la réponse attendue, et ce qu'elle révèle de toi. De quoi transformer 25 pièges en 25 réflexes prêts pour le jour J.

SQL : filtrer, compter, dédoubler.

Ici, le recruteur ne teste pas ta syntaxe. Il vérifie si tes chiffres peuvent être fiables.

1. Un manager veut voir les régions qui ont dépassé 100 commandes ce mois-ci. Tu mets ce filtre dans le **WHERE** ou dans le **HAVING** ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : mettre la condition dans le WHERE, parce que c'est le premier réflexe de filtre qu'on apprend.

Pourquoi ça tente : « plus de 100 commandes » ressemble à une simple valeur de colonne, pas à un résultat calculé.

Réponse attendue : la condition porte sur un total qui n'existe qu'après le GROUP BY. Le WHERE s'exécute avant l'agrégation, il ne peut pas voir ce total. Il faut donc un HAVING.

Signal envoyé : tu connais l'ordre d'exécution d'une requête SQL, pas seulement sa syntaxe apprise par cœur.

```
SELECT region, COUNT(*) AS nb_commandes
FROM commandes
GROUP BY region
HAVING COUNT(*) > 100;
```

2. Tu joins commandes et clients pour calculer un chiffre d'affaires, et le total explose. Que s'est-il passé ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : penser à un bug SQL ou à des données sales dans la table clients.

Pourquoi ça tente : un total plus gros ressemble à un problème de qualité de données, pas à un problème de structure de requête.

Réponse attendue : la jointure porte sur une clé dupliquée côté clients (un même client sur plusieurs lignes) : chaque ligne de commandes est alors comptée en double, triple. On agrège ou déduplique clients avant de joindre.

Signal envoyé : tu ne fais jamais confiance à une jointure sans vérifier le grain avant et après.

```
SELECT COUNT(*) FROM commandes;           -- avant
SELECT COUNT(*) FROM commandes c
JOIN clients cl ON c.client_id = cl.id;     -- apres
```

3. Tu veux compter les clients ayant renseigné un email. COUNT(*) ou COUNT(email) ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : utiliser COUNT(*) par automatisme, parce que c'est la première fonction de comptage apprise.

Pourquoi ça tente : COUNT(*) « marche toujours » pour compter des lignes, donc il semble être le bon réflexe partout.

Réponse attendue : COUNT(*) compte toutes les lignes, NULL compris : il compterait aussi les clients sans email. COUNT(email) ignore les NULL et donne le vrai nombre de clients avec un email renseigné.

Signal envoyé : tu sais qu'un chiffre juste syntaxiquement peut être faux du point de vue métier.

4. On te demande de lister les commandes en double. DISTINCT ou GROUP BY + HAVING ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : appliquer un DISTINCT sur la requête pour « nettoyer » les doublons.

Pourquoi ça tente : DISTINCT semble régler le mot « doublons » en un seul mot-clé, sans détour.

Réponse attendue : DISTINCT masque les doublons dans le résultat, il ne les révèle pas. Pour les identifier, on regroupe sur la clé suspecte puis on filtre les groupes de taille supérieure à 1.

Signal envoyé : tu distingues caché un problème de le diagnostiquer, deux réflexes très différents en entretien.

```
SELECT commande_id, COUNT(*) AS occurrences
FROM commandes
GROUP BY commande_id
HAVING COUNT(*) > 1;
```

5. Tu dois sortir le produit le plus vendu par catégorie. ROW_NUMBER() ou RANK() ?

ÉLIMINATOIRE

Le piège :

Réflexe tentant : se dire que les deux fonctions font « la même chose » et prendre la première qui vient à l'esprit.

Pourquoi ça tente : les deux renvoient un classement croissant, la différence semble cosmétique en dehors des ex-aequo.

Réponse attendue : ROW_NUMBER attribue un rang unique même en cas d'égalité, donc un seul gagnant arbitraire. RANK laisse apparaître les ex-aequo. Pour un top 1 par catégorie fiable, il faut savoir lequel des deux comportements tu veux avant d'écrire la requête.

Signal envoyé : tu anticipes le cas des ex-aequo au lieu de le découvrir en production.

```
SELECT *, ROW_NUMBER() OVER (
  PARTITION BY categorie ORDER BY ventes DESC
) AS rang
FROM produits;
```

À retenir avant l'entretien SQL

Le recruteur ne teste pas ta mémoire de syntaxe, il teste si tu protèges le chiffre.

Ordre d'exécution

WHERE avant, HAVING après le
GROUP BY.

Grain de la jointure

Compare les lignes avant et après un
JOIN.

NULL et doublons

COUNT(*) vs COUNT(colonne), DIS-
TINCT vs GROUP BY.

Excel : au-delà du réflexe RECHERCHEV.

Ici, il vérifie si tu connais les limites de ton outil, pas juste ses fonctions.

6. Ta RECHERCHEV renvoie N/A alors que la valeur existe dans la feuille. Quelle est la cause la plus probable ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : penser à une faute de frappe ou un espace parasite, et recommencer la saisie.

Pourquoi ça tente : une erreur N/A ressemble à un problème de données, donc le réflexe va vers l'orthographe avant la logique de la fonction.

Réponse attendue : RECHERCHEV ne peut chercher que vers la droite de la colonne de référence. Si la colonne à ramener est à gauche de la colonne recherchée, la fonction ne trouvera jamais la valeur, faute de frappe ou pas. La solution : réorganiser les colonnes, ou passer par INDEX/EQUIV, qui ne dépend pas du sens.

Signal envoyé : tu comprends la limite structurelle de l'outil, pas juste son message d'erreur.

7. Tu croises 3 dimensions sur un fichier de 50 000 lignes pour une exploration rapide. TCD ou formule SOMME.SI imbriquée ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : partir sur une formule SOMME.SI, parce qu'elle « reste dans la cellule » et paraît plus contrôlable.

Pourquoi ça tente : une formule donne l'impression de garder la main, alors qu'un TCD paraît être une boîte noire manipulée à la souris.

Réponse attendue (conseil Datafuji, pas une norme chiffrée) : pour une exploration ad hoc à plusieurs dimensions, un tableau croisé dynamique est plus robuste et plus rapide à faire évoluer qu'une formule imbriquée, difficile à relire et à corriger. On garde la formule pour un calcul ciblé et stable, pas pour explorer.

Signal envoyé : tu choisis l'outil selon le besoin, explorer ou figer, pas par habitude.

📌 Coup d'œil

Un tableau croisé dynamique reste modifiable en 2 clics ; une formule imbriquée demande de tout réécrire.

8. Ta formule tourne bien sur ton fichier de test. Le recruteur demande : et si une ligne a une division par zéro en prod ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : répondre que « ça n'arrivera pas », parce que le fichier de test ne contient pas ce cas.

Pourquoi ça tente : on valide sur les données qu'on a sous les yeux, pas sur celles qu'on n'a pas encore vues.

Réponse attendue : on sécurise la formule avec SIERREUR pour intercepter DIV/0! et N/A, et renvoyer une valeur de repli lisible plutôt que de laisser planter un classeur partagé en production. Formule : `=SIERREUR(A2/B2; "n. d. ")`

Signal envoyé : tu penses à la robustesse d'un fichier réutilisé par d'autres, pas seulement au cas que tu as testé.

À retenir avant l'entretien Excel

Le bon outil dépend du besoin (explorer, figer, sécuriser), jamais de l'habitude.

Sens de lecture

RECHERCHEV va seulement vers la droite, INDEX/EQUIV s'en affranchit.

Fichier réutilisable

SIERREUR protège un classeur partagé des erreurs de prod.

Python et pandas : la vitesse qui trahit.

Ici, une boucle de trop et le recruteur sait déjà à qui il parle.

9. Tu calcules une colonne à partir de deux colonnes sur un DataFrame de plusieurs centaines de milliers de lignes. Tu pars sur une boucle avec `iterrows()` ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : écrire `for row in df.iterrows():`, parce que ça ressemble à la logique naturelle « pour chaque ligne, calcule ».

Pourquoi ça tente : la boucle est la structure la plus intuitive quand on vient d'apprendre à programmer, elle « se lit » comme le problème.

Réponse attendue : `iterrows()` est nettement plus lent qu'une opération vectorisée sur de gros volumes, car pandas traite les colonnes en bloc plutôt que ligne par ligne. On écrit une opération vectorisée directe.

Signal envoyé : une boucle ligne à ligne en entretien est un signal d'alerte immédiat sur ta maîtrise de pandas.

```
df['marge'] = df['prix'] - df['cout']
```

10. Après un `merge()`, ton nombre de lignes a augmenté sans que tu t'y attendes. Pourquoi ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : supposer que pandas « a bugué » ou que le fichier source contient une erreur d'export. Pourquoi ça tente : un `merge` est perçu comme une opération neutre qui assemble des données, pas comme une opération qui peut les multiplier.

Réponse attendue : comme en SQL, un `merge` sur une clé non unique d'un des deux côtés duplique les lignes correspondantes. On vérifie avant/après avec `.shape`, on précise le `how`, et on déduplique la clé côté droit si besoin.

Signal envoyé : tu contrôles le résultat d'une opération plutôt que de lui faire confiance par défaut.

```
print(df1.shape, df2.shape)
out = df1.merge(df2, on='id', how='left')
print(out.shape)
```

11. Tu calcules la somme et la moyenne des ventes par client. `agg()` ou `apply()` ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : partir sur `apply()` avec une fonction personnalisée, parce que ça semble plus flexible et plus « maîtrisé ».

Pourquoi ça tente : `apply()` donne l'impression de tout contrôler puisqu'on écrit soi-même la logique, alors qu'`agg()` paraît limité aux cas simples.

Réponse attendue (conseil Datafuji) : pour des agrégations simples (somme, moyenne, comptage), `agg()` est plus direct et plus rapide, car il s'appuie sur les fonctions optimisées de pandas. On réserve `apply()` aux cas où la logique par groupe ne peut vraiment pas s'exprimer avec des agrégats standards.

Signal envoyé : tu ne choisis pas l'outil le plus impressionnant, tu choisis le plus adapté au besoin.

```
df.groupby('client')['ventes'].agg(['sum', 'mean'])
```

12. Une colonne a 15% de valeurs manquantes. Tu fais `dropna()` ou `fillna()` par réflexe ?

ÉLIMINATOIRE

Le piège :

Réflexe tentant : appliquer `dropna()` tout de suite pour « nettoyer » le jeu de données avant d'aller plus loin.
Pourquoi ça tente : supprimer les lignes incomplètes donne l'impression de partir sur des données propres, sans risque apparent.

Réponse attendue (conseil Datafuji) : le choix dépend du sens métier de la valeur manquante, pas d'un réflexe automatique. Si l'absence de valeur est elle-même une information (un client jamais acheteur n'a pas de « dernier achat »), la supprimer fausse l'analyse. On se demande d'abord pourquoi la valeur manque.

Signal envoyé : tu traites la donnée manquante comme un signal métier à comprendre, pas comme un bruit à effacer.

Pourquoi ce chapitre élimine vite

Une boucle ou un merge non vérifié, ça se voit dès les premières lignes de code.

Vectoriser

Jamais de boucle ligne à ligne sur un gros volume.

Vérifier

Comparer `.shape` avant et après un merge.

Comprendre

Le sens métier d'une valeur manquante avant de la traiter.

Statistiques : le chiffre qui ment poliment.

Ici, il regarde si tu doutes du chiffre juste avant de le présenter comme vrai.

13. Le salaire représentatif d'une équipe de 10 personnes dont un manager gagne 3 fois plus que les autres. Moyenne ou médiane ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant :* donner la moyenne, parce que c'est l'indicateur central le plus connu et le plus rapide à calculer.

Pourquoi ça tente : la moyenne est le premier réflexe statistique appris, elle semble être « la » mesure centrale par défaut.

Réponse attendue : la moyenne est tirée vers le haut par le salaire du manager, elle ne représente aucun salarié réel de l'équipe. La médiane, moins sensible aux valeurs extrêmes, donne une image plus fidèle du salaire typique sur une distribution asymétrique.

Signal envoyé : tu sais qu'un chiffre juste mathématiquement peut être trompeur pour décider.

14. Les mois où les ventes de crèmes glacées montent, les noyades montent aussi. Peut-on dire que l'une cause l'autre ?

ÉLIMINATOIRE

Le piège :

Réflexe tentant : chercher un lien direct, glace qui pousse à se baigner ou baignade qui donne envie de glace.
Pourquoi ça tente : le cerveau cherche naturellement une explication causale dès qu'il voit deux courbes évoluer ensemble.

Réponse attendue : corrélation n'implique pas causalité. Ici, un facteur confondant explique les deux : la chaleur estivale. Avant de conclure à une causalité, on cherche s'il existe une troisième variable qui explique les deux évolutions.

Signal envoyé : tu résistes à la première explication séduisante et tu cherches le facteur caché.

📌 Coup d'œil

Corrélation observée : cherche toujours un troisième facteur avant de parler de cause.

15. Un test sur 12 utilisateurs montre que le bouton rouge convertit mieux que le bleu. Tu généralises ce résultat ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : dire oui, le chiffre est là, le rouge convertit mieux, donc on généralise.

Pourquoi ça tente : le résultat existe, il est mesuré, il paraît donc « prouvé », même si la taille de l'échantillon ne le permet pas.

Réponse attendue : un échantillon de 12 personnes est trop petit pour conclure de façon fiable : l'écart observé peut venir du hasard plutôt que d'un vrai effet. On indique la taille d'échantillon nécessaire, ou on demande plus de données avant de généraliser.

Signal envoyé : tu distingues un résultat mesuré d'un résultat fiable.

16. Pour comparer la dispersion de deux séries de notes, écart-type ou variance ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : citer la variance en premier, parce que c'est souvent la formule apprise avant l'écart-type dans les cours.

Pourquoi ça tente : la variance est la brique de calcul de base, elle vient naturellement à l'esprit avant sa racine carrée.

Réponse attendue : l'écart-type s'exprime dans la même unité que la donnée d'origine, contrairement à la variance, exprimée au carré. Pour comparer et communiquer une dispersion, l'écart-type est directement interprétable.

Signal envoyé : tu penses à la lisibilité du chiffre pour celui qui va le lire, pas seulement à sa formule.

À retenir avant l'entretien statistiques

Un chiffre juste peut raconter une mauvaise histoire s'il n'est pas mis en contexte.

Moyenne vs médiane

Regarder la forme de la distribution avant de choisir.

Corrélation

Chercher un facteur confondant avant de parler de causalité.

Échantillon

Vérifier sa taille avant de généraliser.

Dataviz et BI : le graphique qui décide.

Ici, il regarde si un visuel aide à décider, ou s'il trompe, ou s'il noie.

17. Tu compares les ventes de 8 catégories de produits. Camembert ou barres triées ?

PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : partir sur un camembert, le graphique « de répartition » par défaut dans la tête de beaucoup de débutants.

Pourquoi ça tente : le camembert a une image intuitive de répartition, il semble naturel dès qu'on compare des catégories.

Réponse attendue (bonne pratique dataviz, pas une norme chiffrée) : au-delà de 5 ou 6 catégories, l'œil compare mal des angles. Un bar chart trié du plus grand au plus petit rend la comparaison immédiate, même avec 8 catégories.

Signal envoyé : tu choisis le graphique pour la lecture qu'il permet, pas pour son apparence.

18. Ton graphique en barres montre une hausse spectaculaire ; l'axe Y commence en fait à 80 au lieu de 0. Que fais-tu ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : garder le graphique tel quel, il « montre bien » la tendance et rend le rapport plus percutant.

Pourquoi ça tente : un graphique plus impressionnant semble être un meilleur résultat à présenter, surtout sous pression de délai.

Réponse attendue : un axe qui ne part pas de zéro sur un graphique en barres exagère visuellement l'écart, car la longueur des barres encode la valeur. On repart de zéro, ou si l'axe non nul est vraiment justifié, on le signale clairement au lecteur.

Signal envoyé : tu ne sacrifies pas l'honnêteté du chiffre pour un effet visuel plus fort.

📌 Coup d'œil

Un axe tronqué peut multiplier visuellement un écart réel par 4 ou 5 sans toucher au chiffre lui-même.

19. Un manager demande un dashboard qui affiche 20 KPI « pour tout voir d'un coup ». Tu le construis tel quel ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : satisfaire la demande telle quelle et caser les 20 KPI sur un seul écran, pour ne rien oublier.

Pourquoi ça tente : répondre à la demande exacte semble être la posture la plus sûre et la moins conflictuelle.

Réponse attendue (bonne pratique BI) : un dashboard surchargé de KPI dilue l'action possible du lecteur, qui ne sait plus lequel regarder en premier. On distingue les KPI de suivi des KPI actionnables, et on hiérarchise l'écran autour de 3 à 5 indicateurs prioritaires.

Signal envoyé : tu challenges une demande floue au lieu de l'exécuter sans recul.

20. Ton dashboard Power BI met 20 secondes à charger. Ta première idée est de simplifier les visuels ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : retirer des graphiques ou simplifier les couleurs, en pensant que le poids vient de la couche visuelle.

Pourquoi ça tente : le ralentissement est visible à l'écran, donc le réflexe est de traiter le symptôme visible en premier.

Réponse attendue (pratique BI courante) : la lenteur d'un dashboard vient le plus souvent de la requête source ou du modèle de données, pas de la couche de visualisation. On diagnostique d'abord la source avant de toucher aux visuels.

Signal envoyé : tu diagnostiques la cause avant de corriger le symptôme visible.

Ce que le recruteur vérifie vraiment

Un graphique qui trompe ou un dashboard qui noie n'est jamais neutre.

Lisibilité

Des barres triées plutôt qu'un camembert surchargé.

Honnêteté

Un axe qui part de zéro, ou un écart signalé.

Priorité

3 à 5 KPI actionnables avant le détail.

Modélisation et cas métier : au-delà de la requête.

Ici, il ne teste plus l'outil : il teste si tu poses les bonnes questions avant de calculer.

21. Pour un reporting mensuel récurrent, table plate avec toutes les colonnes, ou schéma en étoile ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : choisir la table plate, une seule table, pas besoin de jointure, ça paraît plus simple.

Pourquoi ça tente : une seule table semble plus rapide à requêter et plus facile à comprendre au premier abord.

Réponse attendue : une table plate non modélisée devient un problème dès que le reporting se répète : lourde, redondante, difficile à faire évoluer, incohérences propagées silencieusement. Un schéma en étoile facilite les jointures et rend le reporting fiable et réutilisable.

Signal envoyé : tu penses à la maintenance du modèle, pas juste à la première requête à sortir.

22. Le manager demande le taux de churn du mois. Tu te mets directement à coder la requête ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : se lancer directement dans le calcul, avec sa propre définition implicite du churn.

Pourquoi ça tente : coder tout de suite donne l'impression d'avancer vite et de répondre efficacement à la demande.

Réponse attendue (bonne pratique cas métier) : la définition du churn est ambiguë tant qu'elle n'est pas clarifiée : sur quelle période, à partir de quelle base de clients, qu'est-ce qu'un client « perdu » exactement. On clarifie ces points avant d'écrire la moindre requête.

Signal envoyé : tu ne fabriques pas un chiffre faux vite, tu poses la bonne question avant.

23. Ton taux de conversion a chuté de 5 points cette semaine. Tu commences par chercher une cause métier ? PIÈGE CLASSIQUE

Décodage : *Réflexe tentant* : chercher immédiatement une explication métier extérieure, une mauvaise campagne ou un bug produit.

Pourquoi ça tente : un chiffre qui bouge donne envie de raconter une histoire immédiatement, surtout si le manager attend une explication.

Réponse attendue (bonne pratique analytique) : avant de chercher une cause métier, on vérifie le dénominateur du taux : un changement de périmètre peut faire bouger un taux sans aucun changement réel de comportement.

Signal envoyé : tu vérifies le calcul avant de raconter une histoire dessus.

Pourquoi ce chapitre élimine

Un chiffre construit sur une mauvaise base ne se rattrape pas en aval.

Modéliser en amont

Un schéma en étoile plutôt qu'une table plate qui s'entasse.

Clarifier avant de calculer

Définition, périmètre, dénominateur, avant la moindre requête.

Posture : ce que révèle ta façon de réagir.

Ici, il ne teste plus l'outil : il teste comment tu réagis sous pression.

24. Le recruteur pose une question technique à laquelle tu n'as pas la réponse. Tu dis « je ne sais pas » et tu attends la suivante ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : rester silencieux ou dire juste « je ne sais pas », pour ne pas s'enfoncer avec une mauvaise réponse.

Pourquoi ça tente : le silence semble plus sûr qu'un raisonnement qui pourrait se révéler faux devant le recruteur.

Réponse attendue (conseil Datafuji) : verbaliser ton raisonnement, même incomplet, est mieux perçu que le silence ou le bluff. Tu dis ce que tu sais déjà autour du sujet, la piste que tu explorerais, et où tu irais chercher la réponse.

Signal envoyé : tu montres comment tu réagis face à un trou, pas seulement ce que tu sais déjà par cœur.

25. Le recruteur conteste ton choix technique. Tu changes d'avis tout de suite pour ne pas le contredire ? ÉLIMINATOIRE

Le piège :

Réflexe tentant : céder immédiatement, pour éviter le conflit avec la personne en face.

Pourquoi ça tente : contredire un recruteur donne l'impression de prendre un risque social, alors que céder semble plus diplomate.

Réponse attendue (conseil Datafuji) : tu défends ton choix avec un critère explicite (performance, lisibilité, coût de maintenance), sans t'entêter si l'argument en face est meilleur. Ni céder par réflexe, ni camper sur position.

Signal envoyé : tes choix techniques reposent sur des critères, pas sur une intuition ou l'envie de plaire.

À retenir sur la posture

Ce chapitre ne teste plus l'outil, il teste comment tu réagis sous pression.

Face à l'inconnu

Verbaliser plutôt que se taire ou bluffer.

Face à la contestation

Argumenter avec un critère plutôt que céder ou s'entêter.

25 sur 25. Tu n'as pas besoin de mémoriser ces réponses au mot près. Retiens la logique commune : la réponse tentante est presque toujours la plus rapide à dire, la réponse attendue est celle qui protège le chiffre ou qui montre que tu doutes avant d'annoncer. Reprends chaque chapitre la veille de ton entretien comme une checklist : SQL, Excel, pandas, statistiques, dataviz, modélisation, posture. Sept familles de pièges, un seul réflexe à garder : ralentis d'une seconde avant de répondre.

Fait main, en solo.

Si ce guide te sert avant un entretien, partage-le : il peut débloquent un autre candidat. Et si un piège précis t'a déjà fait trébucher, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

